

HW-01

End to End Delays

1 Overview

This exercise provides hands-on experience with understanding networking delays and their various components.

2 Learning Objectives

- Understand basics of networking delay
- Understand components of delay, such as Transmission, Propagation and Queuing delay

3 Learning Resources

3.1 Network Delays

1. https://media.pearsoncmg.com/ph/esm/ecs_kurose_compnetwork_8/cw/
2. https://media.pearsoncmg.com/ph/esm/ecs_kurose_compnetwork_8/cw/content/interactiveanimations/transmission-vs-propagation-delay/transmission-propagation-delay-ch1/index.html
3. https://media.pearsoncmg.com/ph/esm/ecs_kurose_compnetwork_8/cw/content/interactiveanimations/queuing-loss-applet/index.html

3.2 Docker Containers

1. <https://www.docker.com/products/docker-desktop/>
2. <https://www.docker.com/>

3.3 Git – Distributed Version Control

<https://git-scm.com/downloads>

4 Prerequisites and environment familiarity

4.1 Linux command familiarity

It is assumed that you are familiar with Linux and command line terminal usage.

4.2 Docker Desktop

The majority of our network assignments will be carried out in a docker based environment using Docker Desktop, which is an application environment for your laptop environment that enables running of containerized applications. The Docker Desktop integrates and provides access to a vast ecosystem of docker images via Docker Hub. Using this application, we will create majority of our networking scenarios and understand working of networking protocols.

It is assumed Docker Desktop and git client has been installed on the laptop.

4.2.1 Creating Simple Docker Instance – Test Setup

Open a terminal on the laptop, and run the following command. This should automatically fetch a test “Hello World” Docker image from docker hub, create the instance, output the message “Hello from Docker!” and exit.

```
$ docker container run --rm hello-world
```

The output of text “Hello from Docker!” indicates that your docker setup is working correctly.

4.2.2 Docker Compose to create multi container applications

The entire multi-container application is defined in a single YAML file, which is used by docker-compose. In this file, we specify details specific to containers including their network IP addresses, applications to run etc. Following file provides configuration details of a simple network consisting of 4 hosts.

LAN-4H.yml

All of the required YAML files for this course are available in github. You can download the same using the following command. (assuming git is already installed on your laptop)

```
$ git clone https://github.com/rprustagi/Docker-based-exercises-  
Network-Security.git  
$ cd Docker-based-exercises-Network-Security/yml  
$
```

4.2.3 Create a simple LAN

Consider a simple network as shown below in **Error! Reference source not found..**

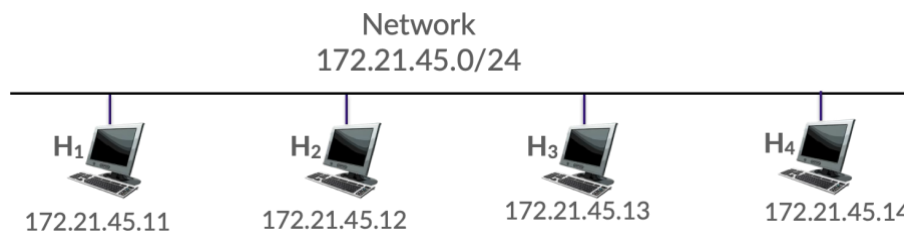


Figure 1: Simple LAN of 4 Hosts

Open your terminal, go to the directory containing YAML files and enter the following command.

Ensure to use appropriate value of \${prefix}, e.g., either arm64 or amd64 as applicable.

```
$ docker-compose -f LAN-4H.yml up -d
```

This will create 4 docker instances as shown below.

```
$> docker-compose -f LAN-4H.yml up -d
```

```
[+] up 5/5
```

```
✓ Network net4-45 Created 0.0s
```

```
✓ Container H1 Created 0.0s
```

- ✓ Container H2 Created 0.0s
- ✓ Container H3 Created 0.0s
- ✓ Container H4 Created 0.0s

\$>

4.2.4 Checking Reachability in a simple LAN

In this network, the hosts H1, H2, H3 and H4 are assigned the IP addresses respectively as 172.21.45.11, 172.21.45.12, 172.21.45.13 and 172.21.45.14. Check reachability of Host H2 from Host H1 using the ping. Similarly, check reachability of other hosts from each other as below. These should result in successful responses.

```
$ docker exec -it H1 ping -c2 172.21.45.12
$ docker exec -it H1 ping -c2 172.21.45.13
$ docker exec -it H1 ping -c2 172.21.45.14
```

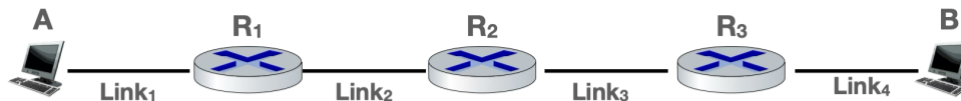
4.2.5 Shutting down simple LAN

Use the following command (in the terminal in the same directory where these docker containers were created) to shut down the network.

```
$> docker-compose -f LAN-4H.yml down --remove-orphans
$>
```

5 Description

5.1 Network Overview



Link	Length (KMs)	Bandwidth (Kbps)
Link 1	2000	400
Link 2	8000	200
Link 3	4000	100
Link 4	6000	80

Router	Queue Size (Num of Buffers)*
R1	a
R2	b
R3	c

* : a, b, c corresponds to last 3 digits of UMBC Id
 e.g. if your UMBC Id is XY00320, then
 a = 3, b = 2, c = 10 (0 is to be taken as 10),

Figure 2: Network of 3 Routers

The assignment is to compute end to end delay when transmitting a series of packets from one host to another host, which are connected via a number of Network Elements (switches/routers). For this exercise consider a network where host A is connected to host B with a network consisting of 3 network elements as shown in Figure 2.

Host A sends 11 packets, each of size 10000 bits without any inter packet gap i.e., transmission of N^{th} packets starts immediately after transmission of $(N-1)^{\text{th}}$ packet is completed. Your task is to compute end to end delay in milli-seconds. The end-to-end delay should take into consideration *Propagation Delay*, *Transmission Delay* and *Queuing Delay*. You can assume that *Processing delay* is zero. For computing propagation delay, consider that speed of electromagnetic communication is $2 \times 10^5 \text{ KM/s}$ (the approximate speed of EM wave in copper medium). Thus, assuming that the first bit of packet starts at time $t=0\text{ms}$, you need to compute the time at which last bit of last packet will be received by Host B. Note that it is not necessary that last packet would be 11^{th} packet. It could be 10^{th} pkt (if 11^{th} packet is discarded by a router when no buffer available) or it could even be 9^{th} packet as well (if both 11^{th} and 10^{th} packets are dropped by routers).

Each router has fixed number of queue buffers (each of size 10000 bits) which excludes the buffer for currently receiving and transmitting packet. When a router receives the packet, it is received in receive buffer. Similarly, when a router transmits the packet, it moves the packet to transmit buffer and then starts transmission. Thus, even if a router has no queue buffer, it can still receive the packet in receive buffer, moves it to transmit buffer and starts transmitting this packet. A router can receive and transmit simultaneously i.e., while transmitting a packet, it can also receive the next incoming packet. Each router uses *Store and Forward* mechanism i.e., it can start transmission only after it has received the complete packet. In case router has no buffer to store the incoming packet(s), it will discard the packet. When a router receives a packet, and transmit buffer is occupied because of previous packet still being transmitted, it will move the packet from receive buffer to queue buffer.

5.2 Delay Computation

For this assignment, consider the network as shown in Figure 2, which also show the bandwidth of each link as well as its length. For our computation, consider that at host A, first packet transmission starts at $t=0\text{ms}$, and your task is to compute its transmission delay and the propagation delay to calculate the time at which it will arrive at router R_1 . The propagation delay of Link-1 is $2000 / (2 \times 10^5) = 10\text{ms}$. Transmission delay of packet 1 is $10000 / (400 \times 10^3) = 25\text{ms}$. Thus, first bit of first packet will arrive at R_1 at $t=10\text{ms}$ and its last bit will arrive at $t=25+10=35\text{ms}$. Thus, router R_1 starts its transmission of first packet at time $t=35\text{ms}$. Since this packet is transmitted immediately upon arrival, there is no queuing delay for this 1^{st} packet in router R_1 .

The 2^{nd} packet transmission at host A will start time $t=25\text{ms}$ and it will be received by router R_1 at $t=60\text{ms}$ (the first bit of 2^{nd} packet arrives at time $t=35\text{ms}$, and last bit at time $t=60\text{ms}$). However, the transmission delay of 1^{st} packet by R_1 is computed as $10000 / (200 \times 10^3) = 50\text{ms}$. Thus, first packet transmission at R_1 will complete at time $t=35+50=85\text{ms}$. Since the 2^{nd} packet arrives at R_1 at time $t=60\text{ms}$, it will be in the first queue buffer from time $t=60\text{ms}$ till time $t=85\text{ms}$ i.e., its queuing delay will be 25ms . The transmission of pkt #2 at R_1 is completed at time $t=135\text{ms}$.

Similarly, 3rd pkt will arrive at R₁ at time $t=85\text{ms}$ it should occupy 2nd queue buffer. However, at this time, R₁ has just completed the transmission of pkt #1 and starts the transmission of pkt #2, moving it from 1st queue buffer to transmit buffer. Thus, upon arrival, pkt #3 is stored in the 1st queue buffer, and queue at R₁ is (pkt #3). The transmission of 3rd pkt at R₁ can start at time $t=135\text{ms}$ and complete at time $t=185\text{ms}$. Thus, queuing delay for 3rd pkt at R₁ will be $135-85=50\text{ms}$.

Your computation should show the entries as per Table 1 for each router and host B. An excel file with some initial filled up entry is provided. All times are in milli seconds. You need to fill remaining entries. The number of buffers at Routers R₁, R₂, and R₃ correspond to 3 digits (*a,b,c*) of your UMBC Id as follows. Take the 3 largest unique digits of your UMBC Id. Digit *a* corresponds to highest of three, digit *b* corresponds to 2nd highest and digit *c* corresponds to the 3rd highest. For example, if your UMBC ID is XY03002, then $a=3$, $b=2$, and $c=0$ (when any values of *a*, *b*, *c* is 0, consider it as 10). Thus, for this UMBC Id, number of buffers at R₁ is 3, number of buffers at R₂ is 2, and number of buffers at R₃ is 10.

5.3 Sample table of computation

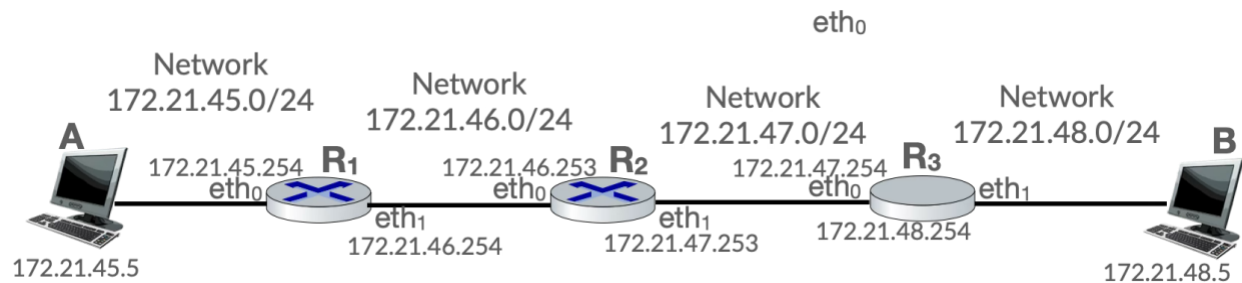
Table 1: Computation Table for end to end delay

Pkt #	Router R1 (a=__)				Router R2 (b=__)				Router R3(c=__)				Host B
	Arr. time	Pkts in Q	Qwait time.	Dep time	Arr.	Pkts in Q	Q time.	Dep time	Arr.	Pkts in Q	Q time.	Dep time	Arr.
1	35	NA	0	35	125		0	125					
2	60	P ₂	25	85	175		50	225					
3	85	P ₃	50	135									
4	110	P ₃ ,P ₄	75	185									
5	135	P ₄ ,P ₅	100	235									
6	160	P ₄ ,P ₅ ,P ₆	125	285									
7	185	P ₅ ,P ₆ ,P ₇	150	335									
8	210	P ₅ ,P ₆ ,P ₇	Dropped	N.A.									
9	235												
10	260												
11	285												

- (Arr.) Arr. time corresponds to arrival time of last bit of the packet
 - Queuing time i.e., how long it remained in the queue, i.e., time between when its last bit arrived and its first bit transmitted. If router has no buffer when packet arrives, it discards this last packet. When the packet is discarded upon arrival, then write "Dropped" in Q time column and N.A. in Dep. Time column.
 - Dep. Time corresponds to transmission start time of first bit of the packet
- Note: Queue time and transmission start time is not applicable to host B.

6 Implementation and Experiential Learning

6.1 Docker Based Network Setup



Consider the network as shown in Figure 3. This network can be created using the docker compose file LAN-4R2H.yml, as follows.

```
docker-compose -f LAN-3R2H.yml up -d
```

6.2 Implementation of propagation and transmission delay and queue buffer size.

The link HA-R1 is 2000KM with propagation delay of 10ms and bandwidth of 400Kbps, and thus for a packet of size 1250 bytes (10000 bits), the transmission delay is 25ms. Host A has no queue buffer limits and thus implement the propagation and transmission delay with following commands.

6.2.1 Configuration at Host A

Login to host HA

```
$> docker exec -it HA bash
```

Queue setup

```
root@HA:/# tc qdisc add dev eth0 root handle 1: htb default 10
```

Add transmission delay by defining transmission bandwidth of 400Kbps

```
A># tc class add dev eth0 parent 1: classid 1:10 htb rate 400kbit ceil 400kbit
```

Add propagation delay of 10ms

```
A># tc qdisc add dev eth0 parent 1:10 handle 10: netem delay 10ms
```

6.2.2 Configuration at Router R1

Configure the propagation and transmission delay related configurations for packets that is forwarded by R1 to R2. In the Figure 3, this is interface eth1. Use the interface as applicable to your setup.

Login to host R1

```
$> docker exec -it R1 bash
```

Queue setup

```
root@R1:/# tc qdisc add dev eth1 root handle 1: htb default 10
```

Add transmission delay by defining transmission bandwidth of 200kbps, and specify the queue buffer of size 3.

```
R1># tc class add dev eth1 parent 1: classid 1:10 htb rate 200kbit ceil  
200kbit
```

Add propagation delay of 40ms and buffer queue size of 3 (value of queue buffer size a). Use the value as applicable to you i.e., replace the value of 3 by value of a as applicable.

```
R1># tc qdisc add dev eth1 parent 1:10 handle 10: netem delay 40ms limit 3
```

6.2.3 Configuration at Router R2, R3

Configure Routers R2 and R3 similar to R1.

6.3 Generating Traffic.

Given that queue buffer sizes are 3(value of a), 2(value of b), and 10(value of c) as per computation derived in Table 1, when a series of packets are sent continuously, some of the packets, e.g., from #7, #8, #9 and #10 will be lost. This can be implemented by sending a burst of more than 10 packets of ping traffic having size slight less than 1250 bytes (e.g., 1232 bytes to account of Network Layer encapsulation overheads etc.). This can be done by issuing following command on host A.

```
A># ping -l 10 -c 20 -s 1232 172.21.48.5
```

The option `-l 10` (small L) implies send first 10 packets without waiting for response. After that it will wait for response or timeout before sending the next packet. This command sends a total of 20 ping packets, each with payload size of 1232 bytes to host B.

6.3.1 Packet Capture Analysis at Host A

Run a tcpdump on host A and analyze the transmission time of first 10 packets (you may need to open another terminal window, go to host A on the Docker, and run the above ping one more time, and you'll see the tcpdump output occurring). These should be spaced at an interval of approx. 25ms (transmission delay). The time is approx. 25ms to account of system processing and computation variations. A sample invocation is shown below

```
# tcpdump -#n -i eth0 src 172.21.45.5 and icmp  
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode  
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes  
1 02:35:17.248608 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,  
seq 1, length 1240  
2 02:35:17.248686 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,  
seq 2, length 1240
```

```

3 02:35:17.249309 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 1, length 1240
4 02:35:17.249318 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 2, length 1240
5 02:35:17.267620 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 3, length 1240
6 02:35:17.267771 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 3, length 1240
7 02:35:17.293680 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 4, length 1240
8 02:35:17.293929 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 4, length 1240
9 02:35:17.321024 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 5, length 1240
10 02:35:17.346365 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 6, length 1240
11 02:35:17.369535 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 7, length 1240
12 02:35:17.395890 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 8, length 1240
13 02:35:17.423652 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 9, length 1240
14 02:35:17.447282 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 10, length 1240
15 02:35:18.256747 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 11, length 1240
16 02:35:19.254138 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 12, length 1240
17 02:35:20.257932 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 13, length 1240

```

From above, it can be seen that time diff between captured pkt #8 with ICMP Seq 4 (time 02:35:17.293929) and pkt #9 with ICMP seq 5 (time 02:35:17.321024) is about 28ms, and same is the case for subsequent packets. The initial few packets do not show such transmission delay as system initially adjusts to deal with queue filling and transmission buffer setup. This shows transmission delay of >25ms between all packets at Host A.

6.3.2 Packet Capture Analysis at Router R1.

Run *tcpdump* on router R1 and analyze the transmission time of first few packets by host A. These would be spaced at interval of approximately 50ms (transmission delay). A sample invocation is shown below

```

R1># tcpdump -#n -i eth1 src 172.21.45.5 and icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
1 02:35:17.293639 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 1, length 1240
2 02:35:17.293670 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 2, length 1240
3 02:35:17.293860 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 1, length 1240
4 02:35:17.293903 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 2, length 1240

```



```

5  02:35:17.331683 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 3, length 1240
6  02:35:17.384027 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 4, length 1240
7  02:35:17.434556 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 5, length 1240
8  02:35:17.484451 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 6, length 1240
9  02:35:17.537544 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 8, length 1240
10 02:35:17.587395 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 10, length 1240
11 02:35:18.301028 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 11, length 1240
12 02:35:19.297478 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 12, length 1240
13 02:35:20.299214 IP 172.21.45.5 > 172.21.47.5: ICMP echo request, id 48,
seq 13, length 1240

```

Time difference between captured pkt #5 and subsequent packet is about ~50ms. For example, time diff between pkt #5 and #6 is (02:35:17.384027 - 02:35:17.331683 = ~53ms). Similar to host A, initial few packets do not show such delay as system initially adjusts to deal with queue filling and transmission buffer setup. This shows transmission delay of >50ms between all packets at Host A. Similarly, a careful analysis of pkt #9 (ICMP Echo request seq 8) and pkt #10 (ICMP Echo request seq 10) shows that packets corresponding to ICMP Echo request seq 7 and ICMP Echo request seq 9 are missing. These are dropped due to limited buffer (queue buffer size of $n=3$) at router R1. This is expected since route R1 is receiving at faster rate than it can transmit. The bandwidth on incoming link 400Kbps, which is twice of 200Kbps, the transmission bandwidth of outgoing link. Thus, when queue buffer becomes full, it simply drops the packet. Packets starting from to ICMP Echo request seq 11 are transmitted at interval of 1s, and thus do not face any queue buffer situation.

6.3.3 Packet Capture Analysis at Router R2.

Run tcpdump on router R2 and analyze the transmission time of first few packets by host A. These would be spaced at interval of approx. 100ms (transmission delay). The details of packet capture at R2 are not shown here.

Note: Packet captures A, R1, R2 and R3 should be done simultaneously in different windows for consistency of results.

7 Assessment

Please submit the following

1. A file Readme.txt file providing following:
 - a. UMBC Id
 - b. Name
 - c. Summary of your learning
 - d. Challenges faced in doing the assignment and how did you address it.

- e. Approximate time (in hours) taken to do the assignment.
- 2. An excel file or pdf scan of table computation on paper. An excel file is provided showing some initial computation.
- 3. If any packet is dropped in the computation table, explanation of why it was dropped.
- 4. Commands issued at host A, R1, and R2 and corresponding packet capture showing the transmission delay and packet loss (provide an output or a screenshot)
- 5. Explanation of packet delays and packet loss in the network implementation. It is possible that there may be some difference between packet loss shown in the excel file and packet loss experienced in a live setup.
- 6. For tcpdump packet, capture, either attach the outputs or screenshots.

8 Evaluation: 20 marks

- i. `Readme.txt` with proper details: 2 marks
- ii. Computation of delays is shown in table with explanations: 10 marks. The provided sample excel file can be used and submitted after entering values.
 - a. 1 mark for computation details of each packet.
- iii. Network implementation and demonstration of delays: 8 marks
 - a. 2 marks each for correct `tc` commands to setup transmission delay, propagation delay and queue buffer size at host A, Router R1, R2 and R3, and explanation of packet capture and identifying packet loss at router R1, R2 and R3.

9 Note

Any plagiarism activity will result in penalties of being awarded 0 marks.

<end of HW-01>